

# DODO AI 平台使用手册

## 1. DODO 平台介绍

DODO 平台是一款覆盖 AI 模型的开发、训练、评估、测试全流程的 AI 开发平台。平台支持深度学习、强化学习、垂域大模型微调等各领域模型全流程开发，内置丰富的算法库和模型微调机制。DODO 平台基于云原生技术开发，对集群算力统一编排调度，支持多种部署方式（一体机、私有云、公有云）。

## 2. 注册与登录

首先参照附件一 OpenVpn 配置，配置好 VPN 后，访问页面 <http://marl.dodo-ai.cn>，点击注册账号。



### 登录

很高兴再次见到您

\* 用户名

167281

\* 密码

167281

☒ 记住密码

登录

飞书登录

1.点击注册账号

注册账号



在注册页面，填写用户名、设置密码，填写课程统一下发的邀请码，此处请填写真实的学生姓名和学号



## 注册

\*用户名  
请输入用户名

\*密码  
请输入密码

\*确认密码  
请再次输入密码

\*邀请码  
请输入邀请码

由助教统一下发

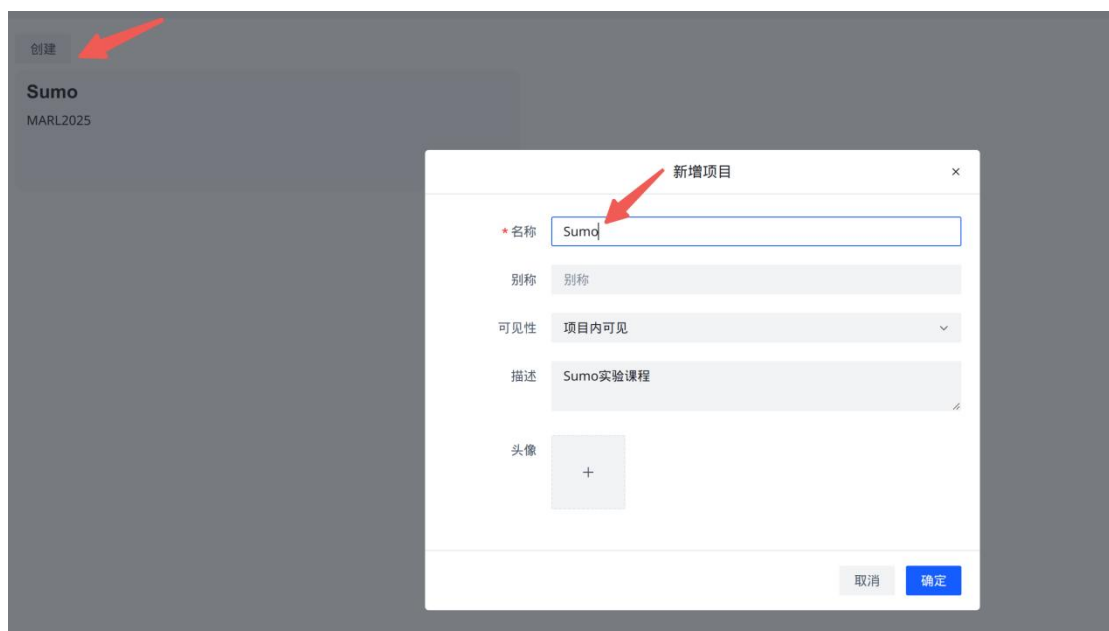
注册 取消



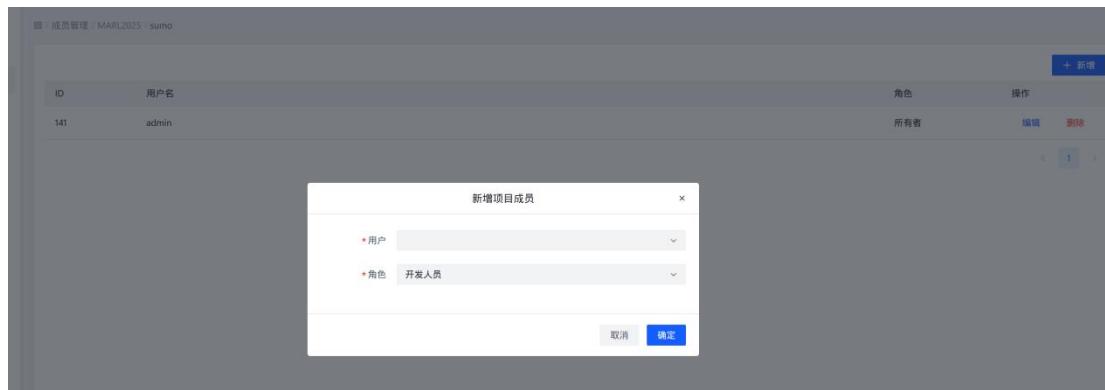
注册完毕后，等待经过审核后，正常登录使用。

## 3. 项目管理

在工作空间 MARL2025 【项目】页面，点击创建按钮，填写项目名称，设置可见性。



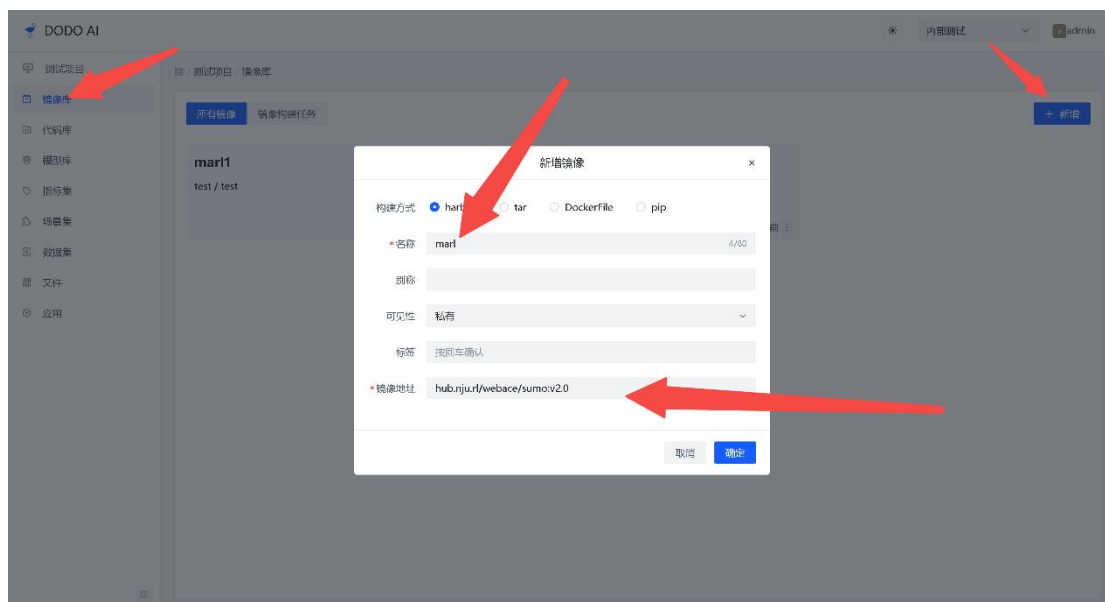
根据项目和团队需要，选择相应的其他用户进入本项目中，形成开发小组。



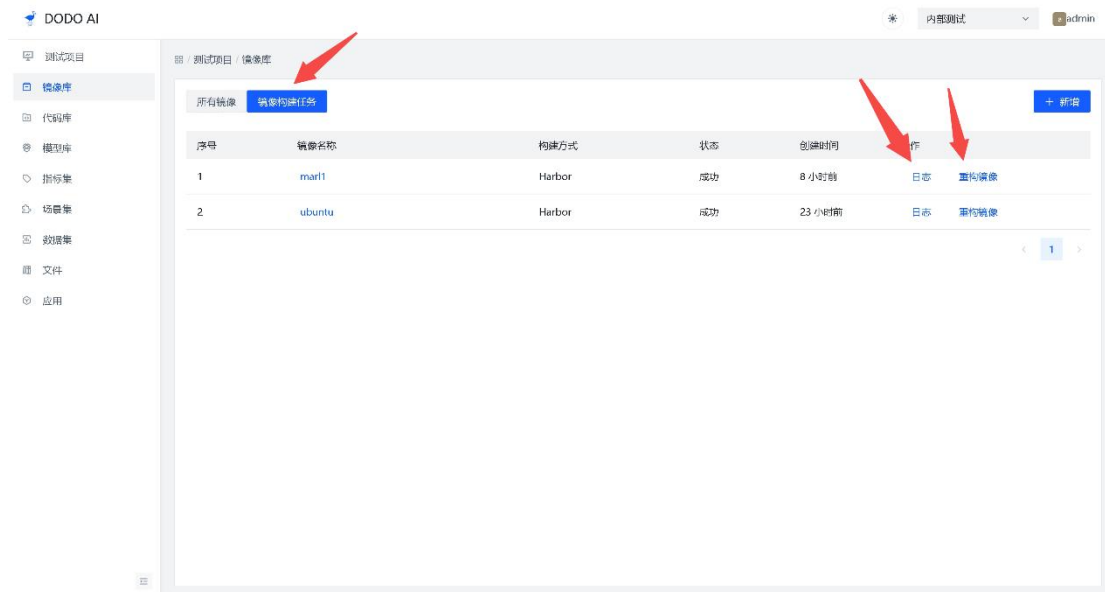
## 4. 镜像管理

### 4.1. 创建算法镜像

在项目的左侧菜单栏，点击镜像库菜单项，在镜像库页面，点击新增按钮，在弹出的对话框中填写镜像名称，上传镜像地址。



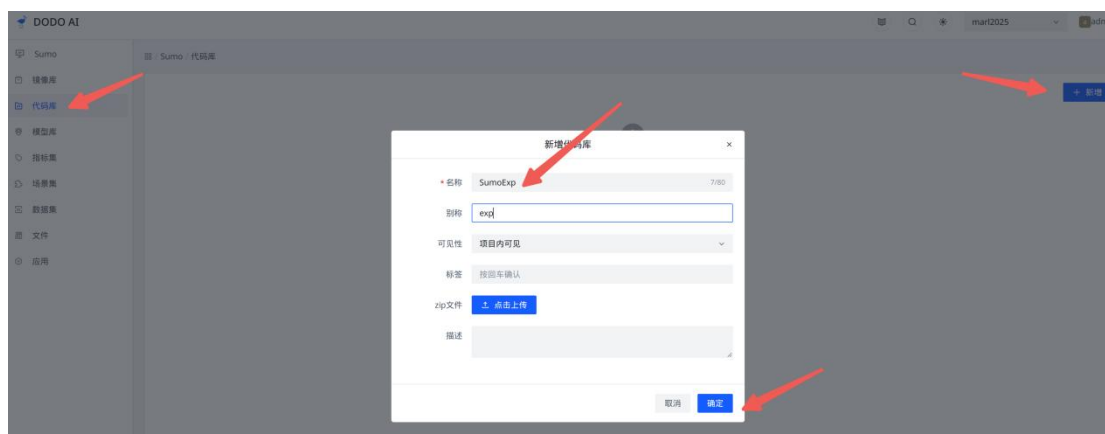
在镜像库菜单项中，点击镜像构建任务菜单项，在镜像构建页面，点击日志按钮可查看镜像状态信息。如果镜像构建识别可点击重构镜像。



## 5. 代码管理

### 5.1. 创建代码 Repo

在项目的左侧菜单栏，点击代码库菜单项，在代码库页面，点击新增按钮，在弹出的对话框中填写代码 repo 名称，上传代码（可选）。



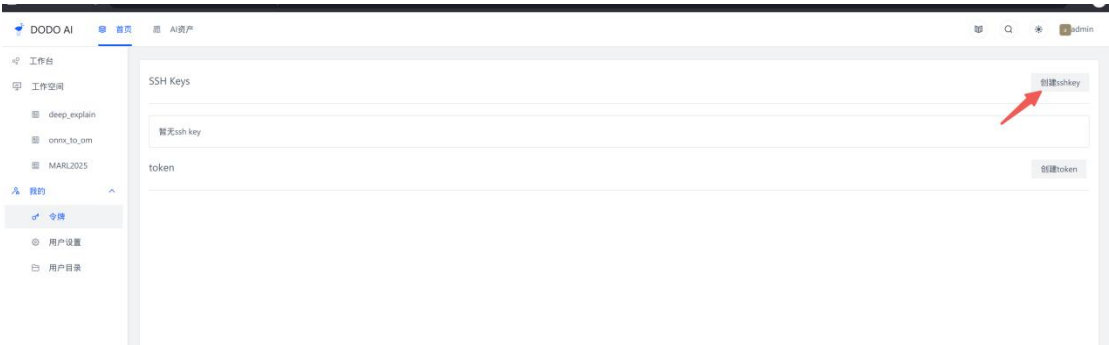
### 5.2. 提交算法代码

提交代码有两种方式，一种是通过 git 来提交，一种是通过页面上传来提交。

### 5.2.1. 通过 git 提交

#### 5.2.1.1. 配置公钥

在我的/令牌页面，点击创建 SSH-KEY 按钮。



在弹出的对话框中，配置 SSH-KEY，点击我的配置，添加本地的公钥。

创建SSH Key

\* 标题

请输入标题

\* key类型

\* 公钥

Begins with 'ssh-rsa', 'ssh-dsa', 'ecdsa-sha2-nistp256', 'ecdsa-sha2-nistp384', 'ecdsa-sha2-nistp521', 'ssh-ed25519', 'sk-ecdsa-sha2-nistp256@openssh.com', 'sk-ssh-ed25519@openssh.com'.

过期时间

请选择日期

取消

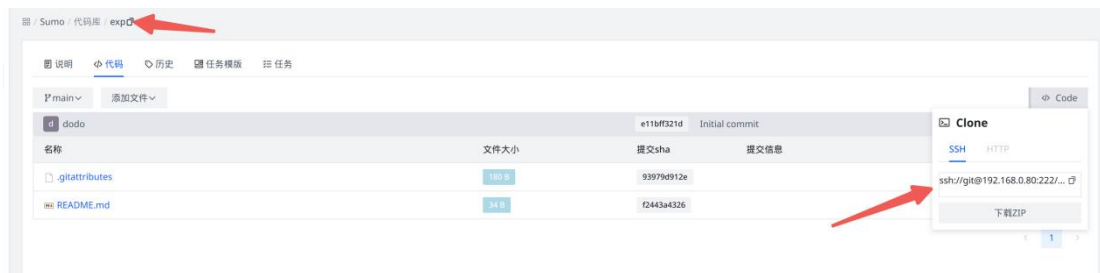
确定

#### 5.2.1.2. 提交代码

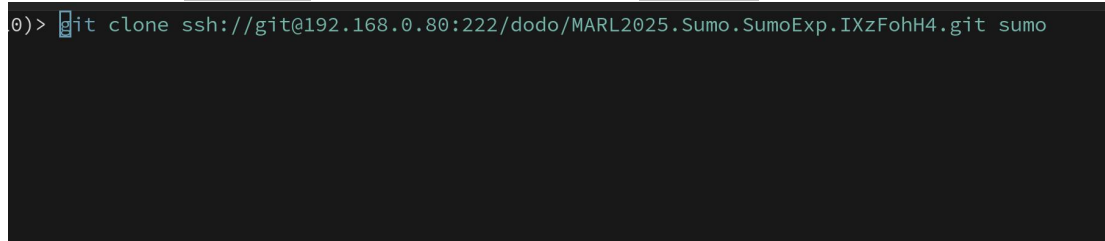
在代码详情中，点击 code 按钮，查看 GIT SSH 路径。根据 git repo URL，提交

自己本地代码。

ssh://git@192.168.0.80:222/dodo/MARL2025.Sumo.SumoExp.IXzFohH4.git

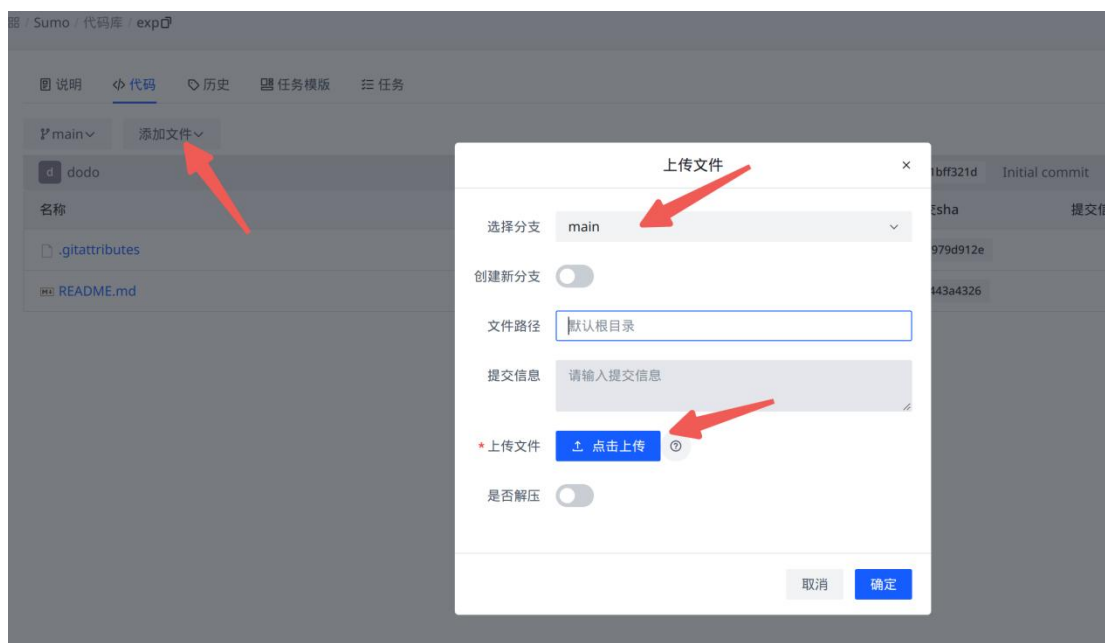


在本地通过 `git clone` 到本地，然后提交通过 `git push` 提交代码。



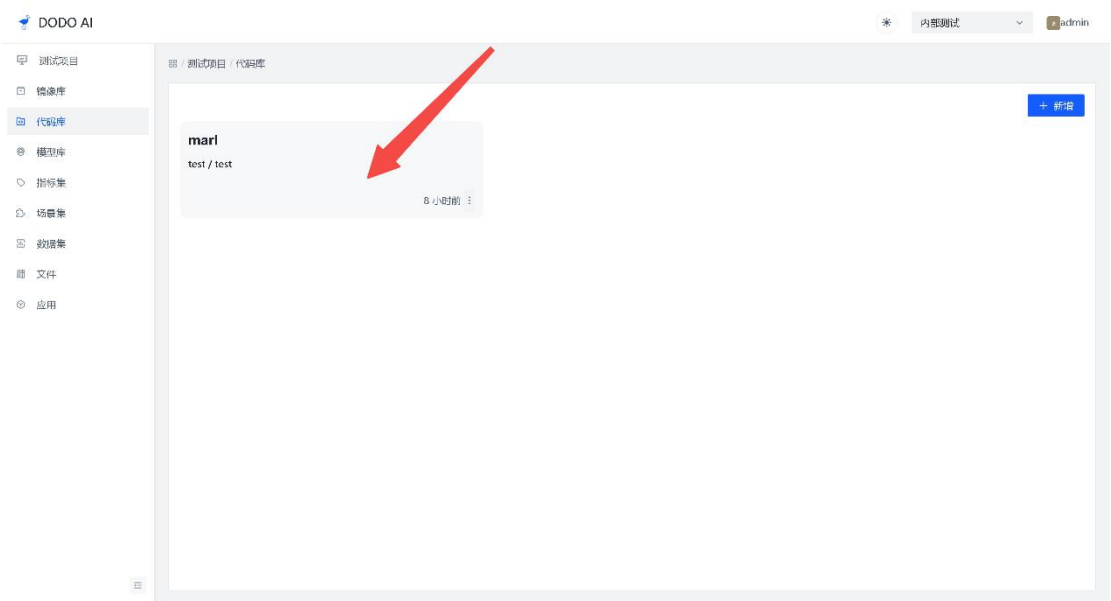
## 5.2.2. 通过页面上传

用户也可以在代码仓库，点击添加文件按钮，在对话框中，选择文件。

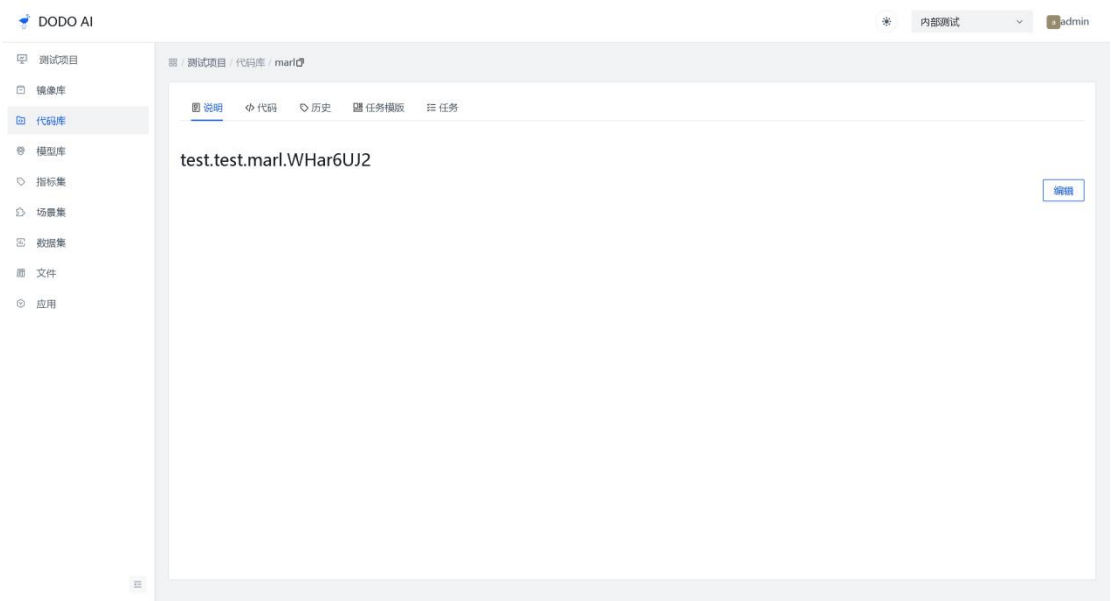


### 5.2.3. 算法修改页面

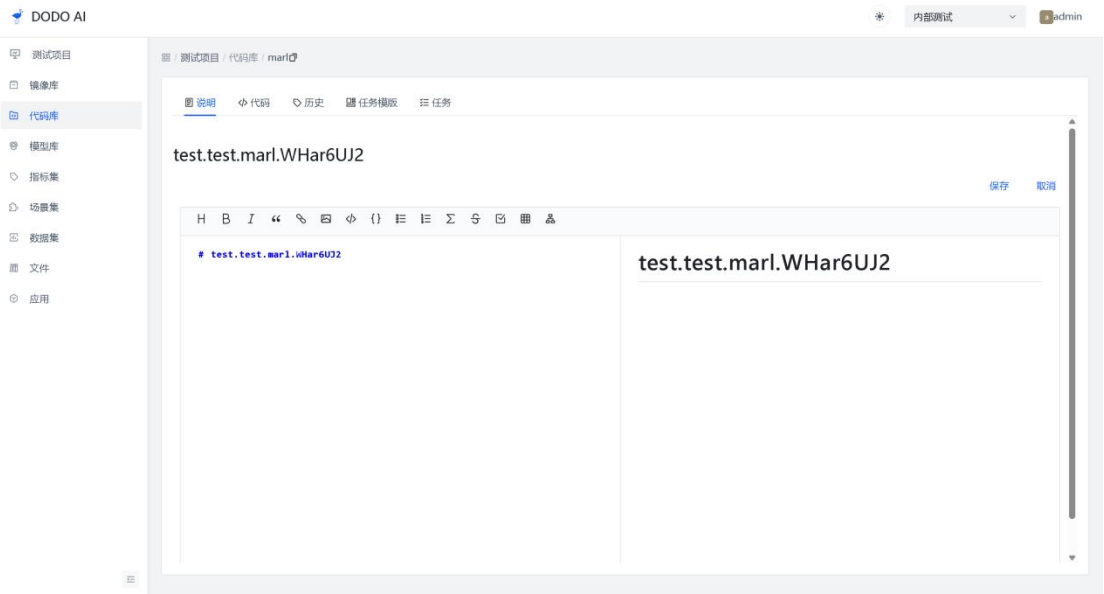
在代码库菜单项中，点击代码菜单项。



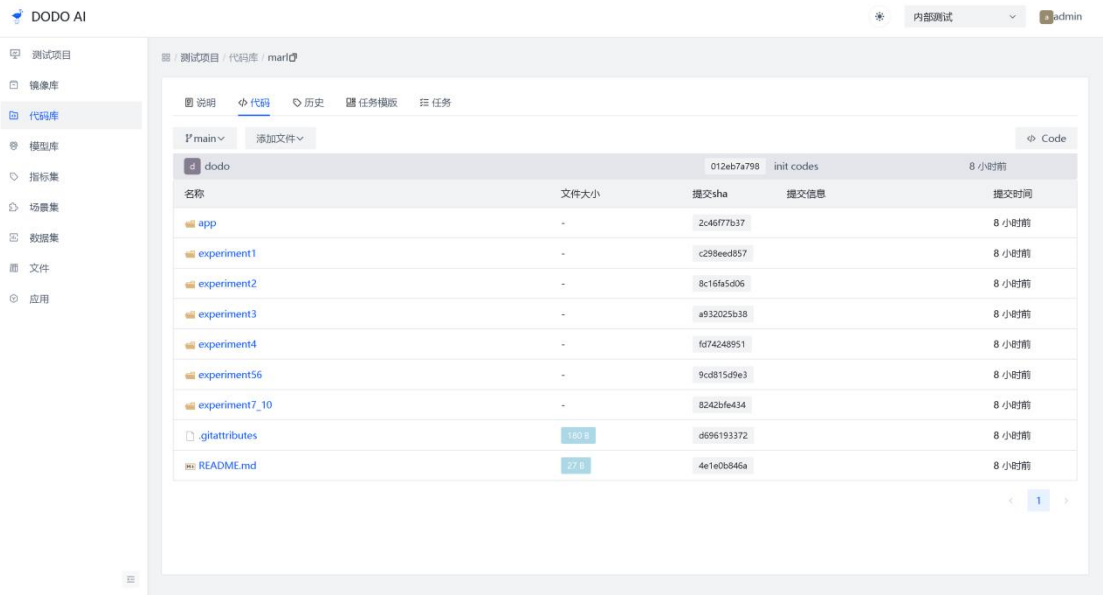
在代码详情页面，包括代码说明、代码、历史、任务模板、任务子菜单项。



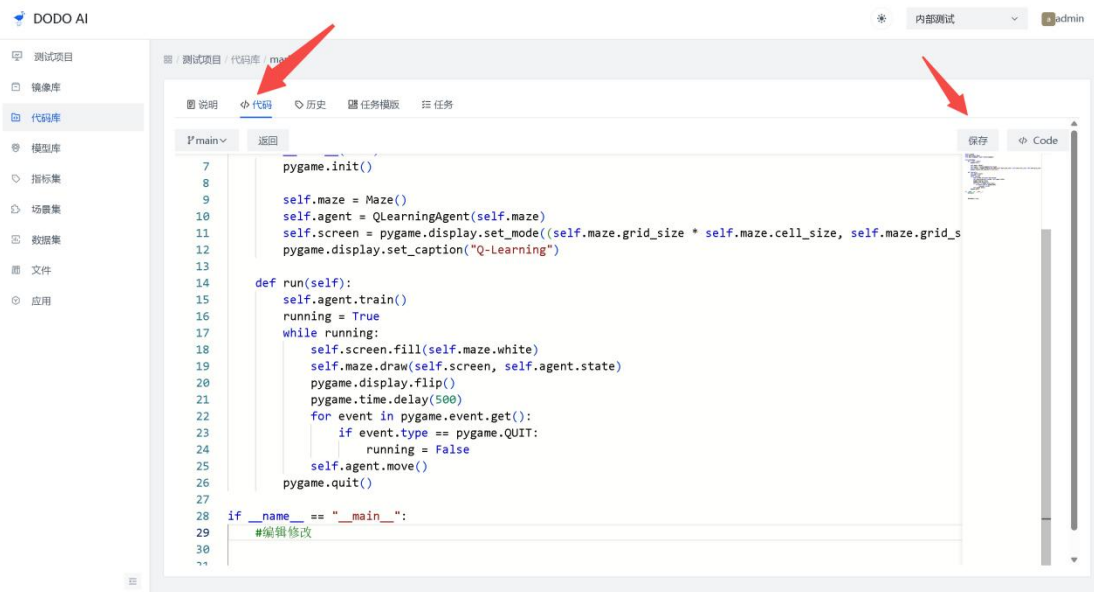
说明页面，可描述算法（xxx 算法的具体名称或用途）



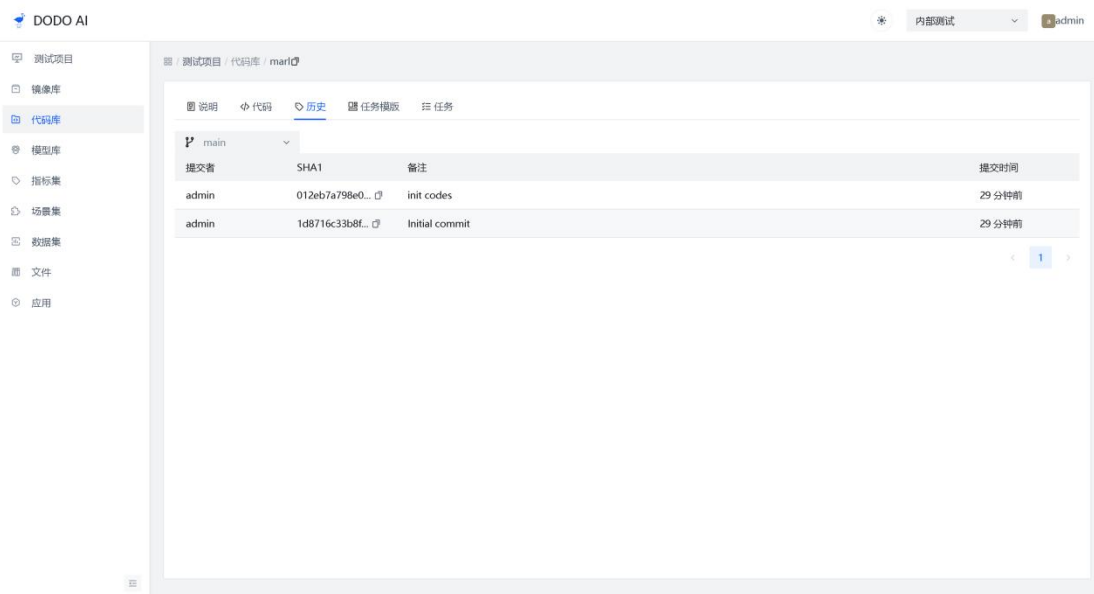
代码页面，该目录作为主目录，包含各类实验，每个子目录针对一种算法分类（如 PPO、DQN、MAPPO 等）。



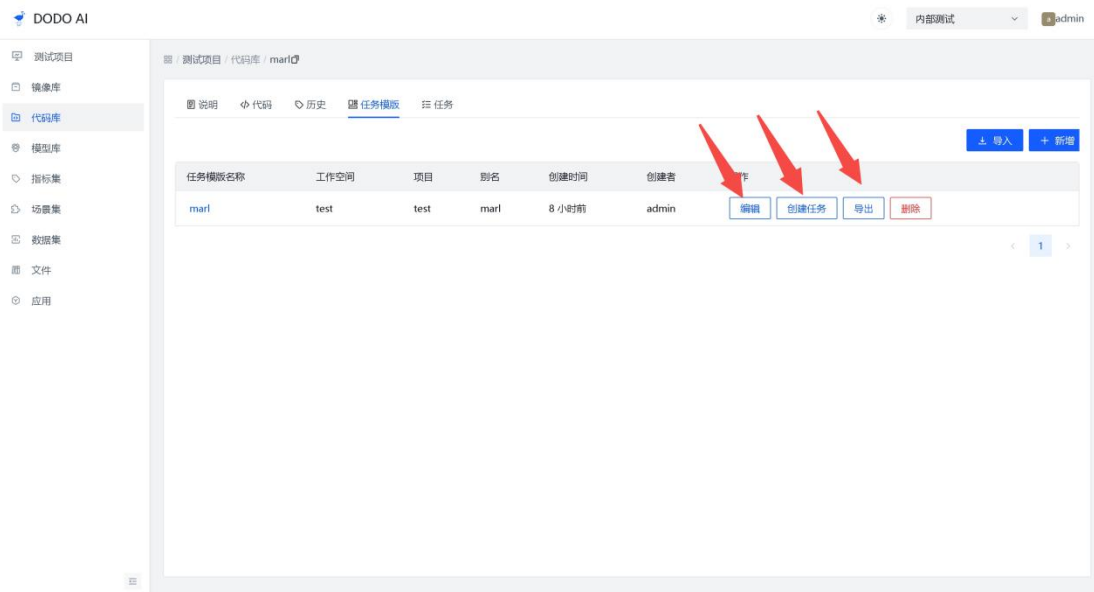
用户可进入指定目录下的 xxx.py 文件，对代码进行编辑和修改。完成修改后，点击“保存”按钮即可更新并保存最新的代码内容。



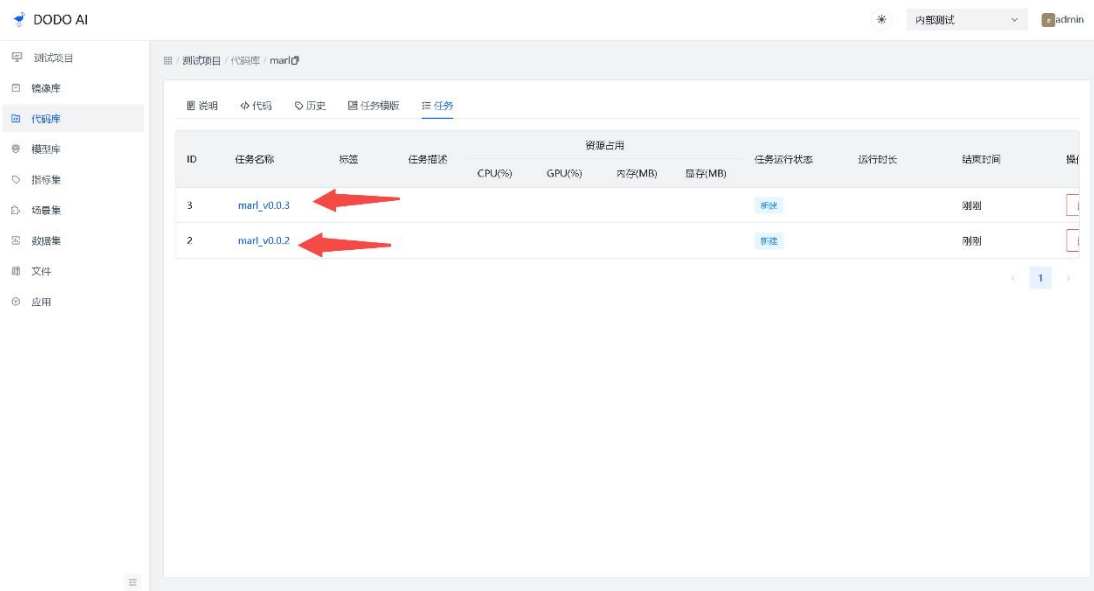
用户点击历史菜单项可查看用户提交代码的信息，包括（提交者、提交时间等）。



用户点击任务模板菜单项可编辑、创建任务等。



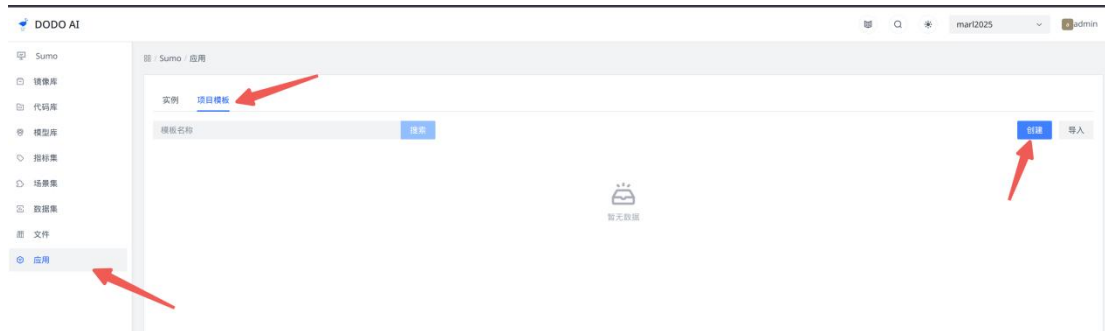
用户点击任务菜单项可查看创建的任务列表。



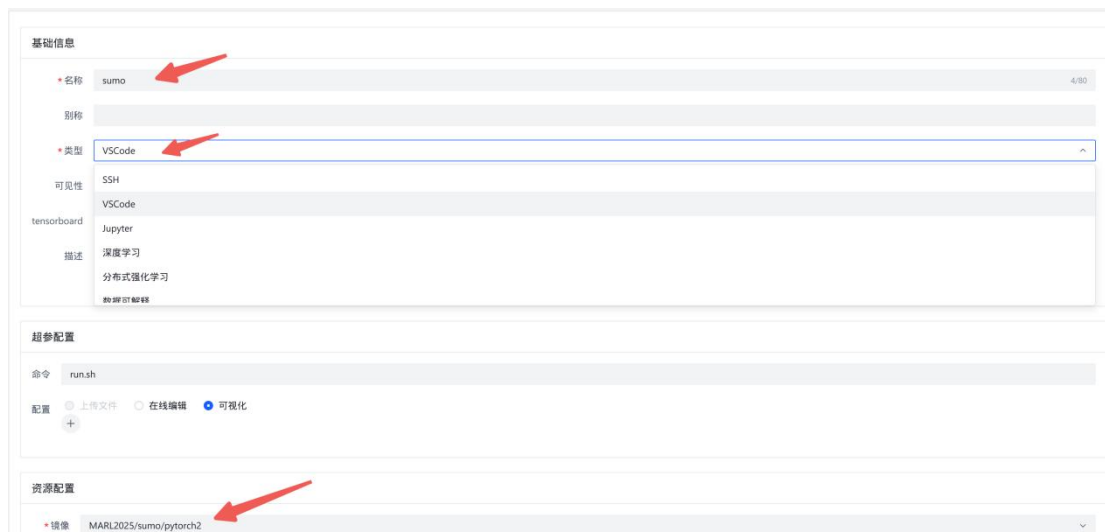
## 6. 实验

### 6.1. 创建实验模板

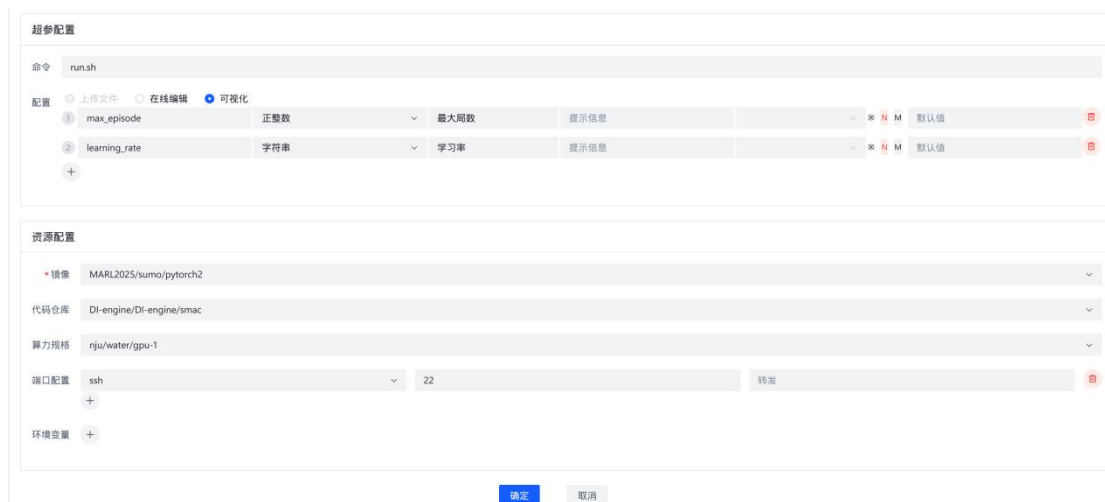
在项目左侧菜单中，点击应用项，在应用页面中，点击创建按钮，创建应用模板。



在创建页面中，填写模板名称，选择镜像，**设置启动命令**，**选择对应的代码库**。

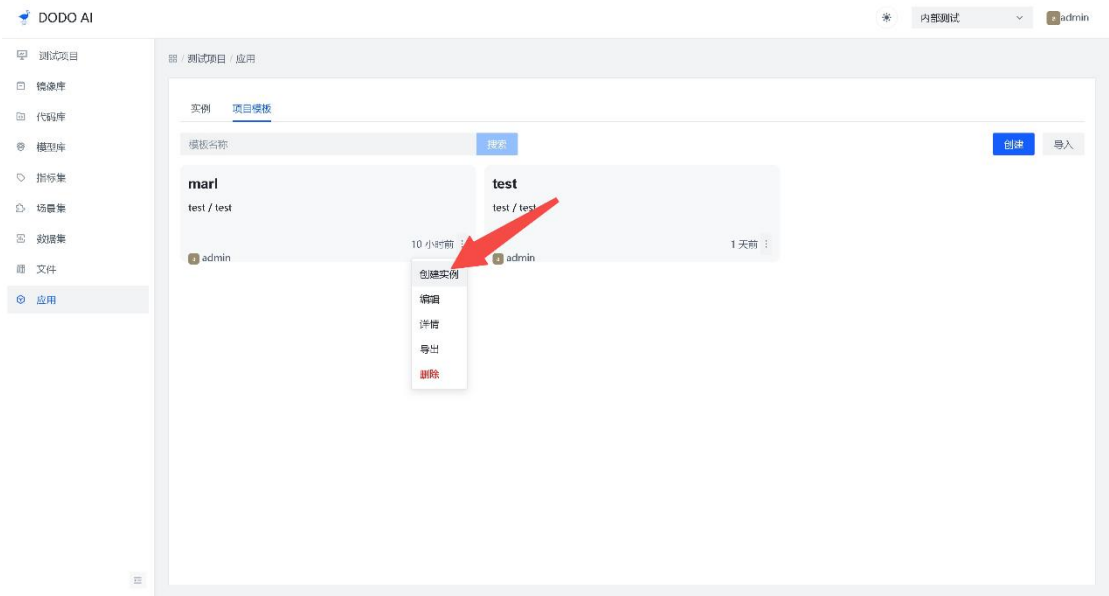


选择算法代码库、设置超参配置、设置端口转发、选择算力规格等。

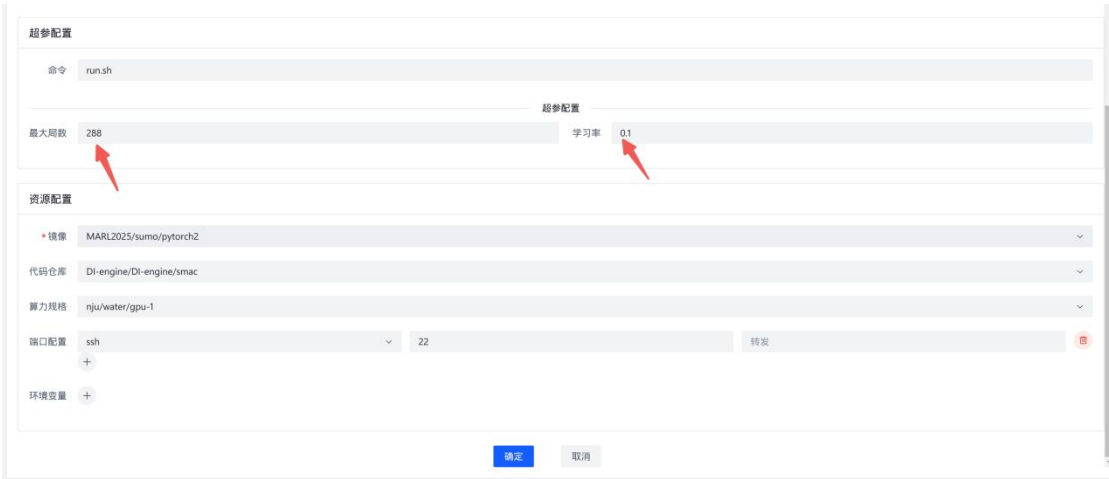


## 6.2. 创建实验

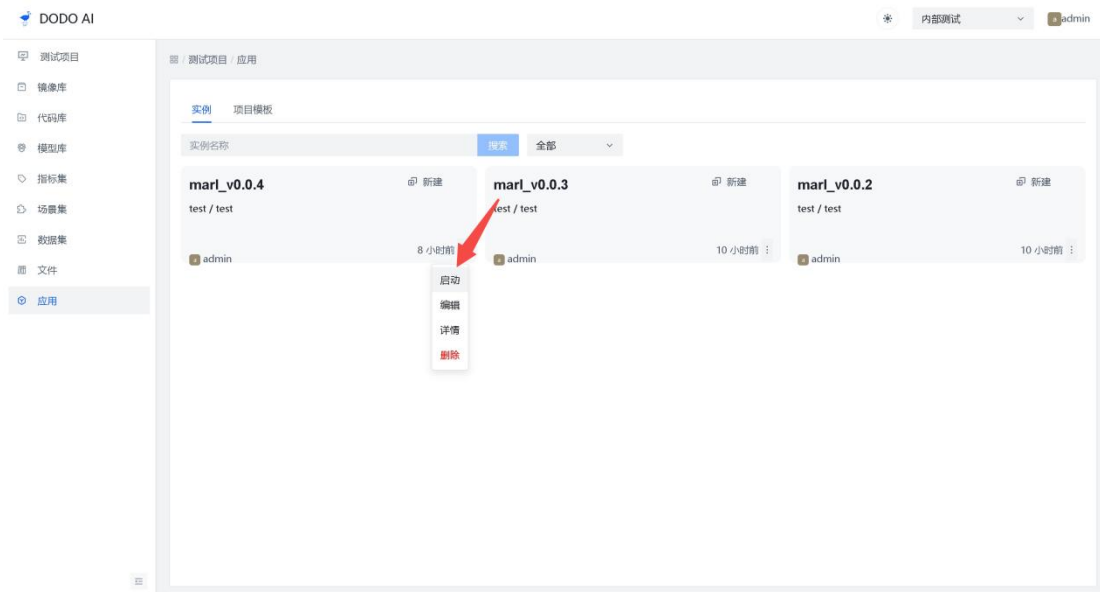
选择实验模板，点击创建实例。



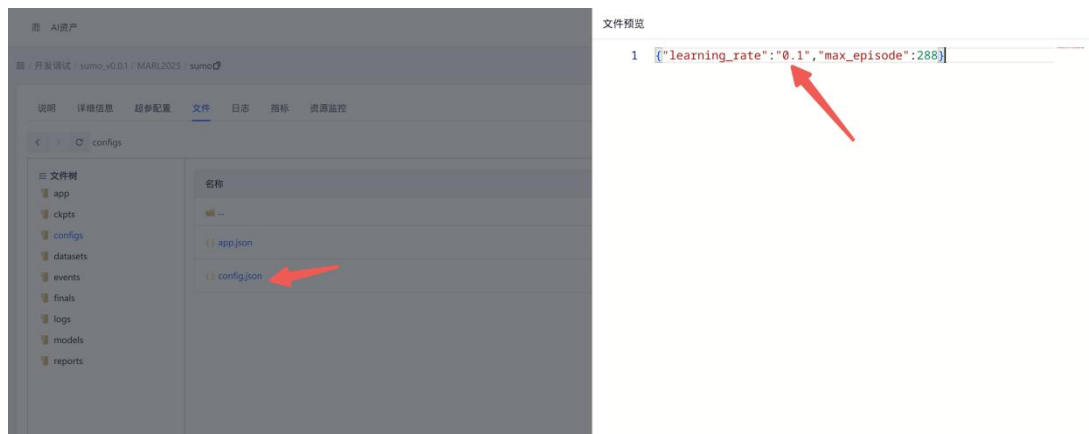
在设置页面，设置相应超参数，比如设置最大局数和学习率。



在任务页面，点击启动按钮开始训练。

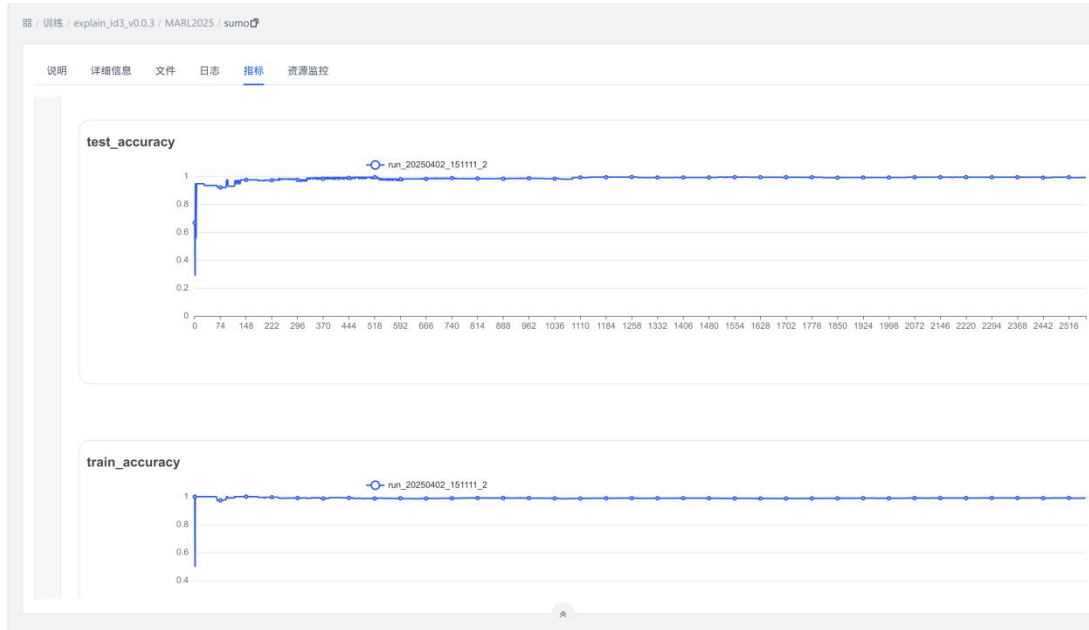


进入任务详情页面后，算法在 `/task/configs/config.json` 中可以读取加载这些参数，路径说明，在任务启动后，git 仓库中的代码会被 pull 到容器的 `/task/app` 目录下。



### 6.3. 实验跟踪

如下图所示，用户可通过 `dodo.log` 方式，跟踪训练过程。



使用方式如下，初始化

```
import dodo

run = dodo.init(
    # Set the name where this run will be logged
    # name="test1",
    # Track hyperparameters and run metadata
    config={
        "lr": lr,
        "epochs": epochs,
    },
)
```

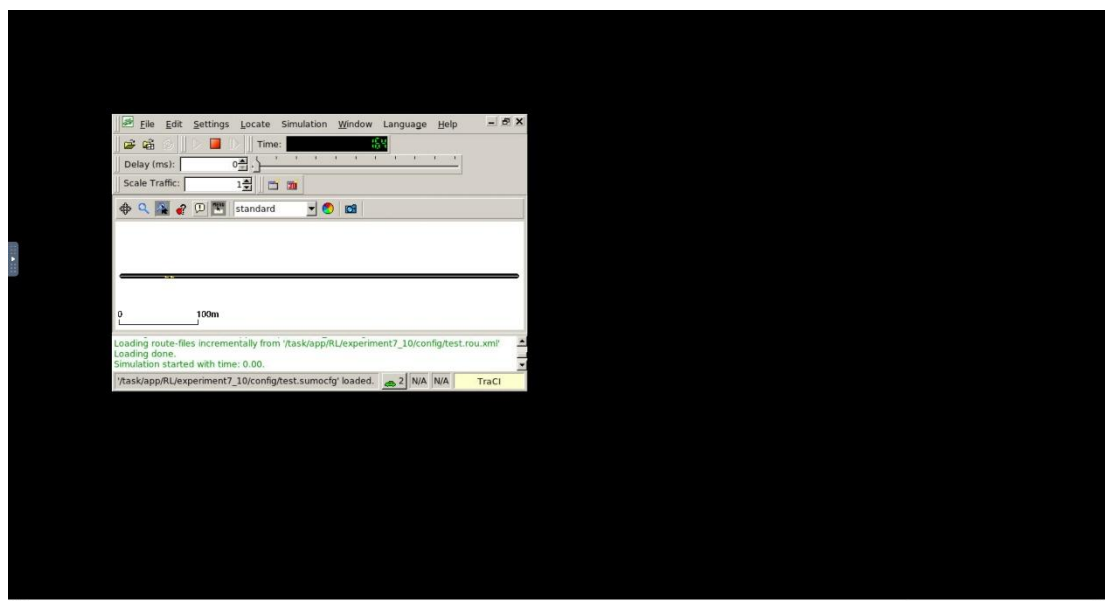
记录训练和测试精度

```
for i in range(1, len(self.x_train) + 1):
    self.model.fit(self.x_train[:i], self.y_train[:i])
    train_pred = self.model.predict(self.x_train[:i])
    test_pred = self.model.predict(self.x_test)
    train_accuracy = accuracy_score(self.y_train[:i], train_pred)
    test_accuracy = accuracy_score(self.y_test, test_pred)
    train_accuracies.append(train_accuracy)
    test_accuracies.append(test_accuracy)

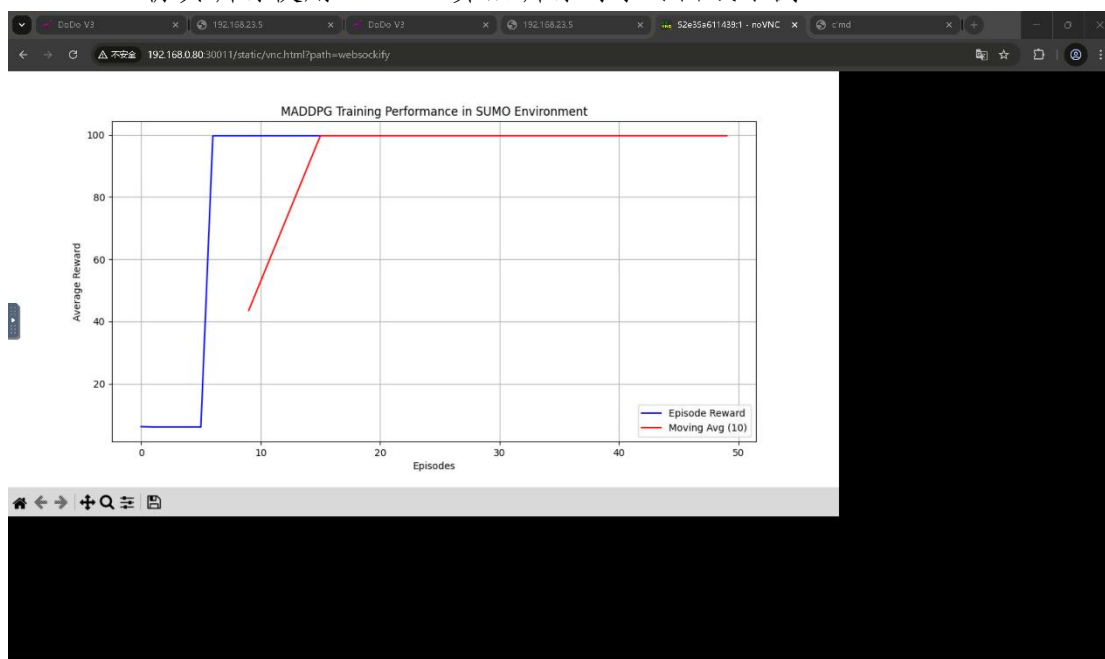
# 使用 TensorBoard 记录训练和测试精度

tf.summary.scalar('train_accuracy', train_accuracy, step=i)
tf.summary.scalar('test_accuracy', test_accuracy, step=i)
dodo.log({"train_accuracy": train_accuracy, "test_accuracy": test_accuracy})
```

使用 sumo 仿真训练过程示例：



sumo 仿真训练使用 MADDPG 算法训练的学习曲线示例：

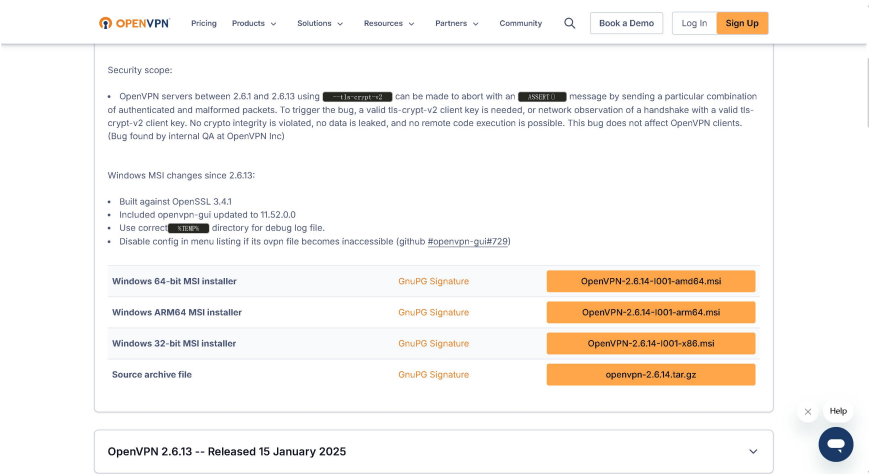


训练过程的日志输出。

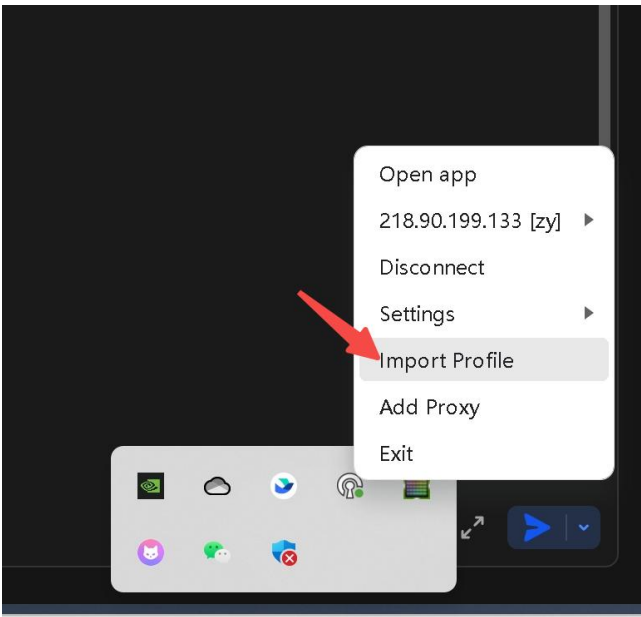
```
仿真中发现的车辆: ['car_1']
Episode 28/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 29/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 30/50: Steps=363, Average Reward=50.2500
模型已保存到 maddpg_checkpoint_ep30.pt
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 31/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 32/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 33/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 34/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 35/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 36/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 37/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 38/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 39/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 40/50: Steps=363, Average Reward=50.2500
模型已保存到 maddpg_checkpoint_ep40.pt
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 41/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 42/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 43/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 44/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 45/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 46/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 47/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 48/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 49/50: Steps=363, Average Reward=50.2500
  Retrying in 1 seconds
仿真中发现的车辆: ['car_1']
Episode 50/50: Steps=363, Average Reward=50.2500
模型已保存到 maddpg_checkpoint_ep50.pt
```

# 附件一、OpenVPV 配置

在网站下载 openVPN（<https://openvpn.net/community-downloads/>）

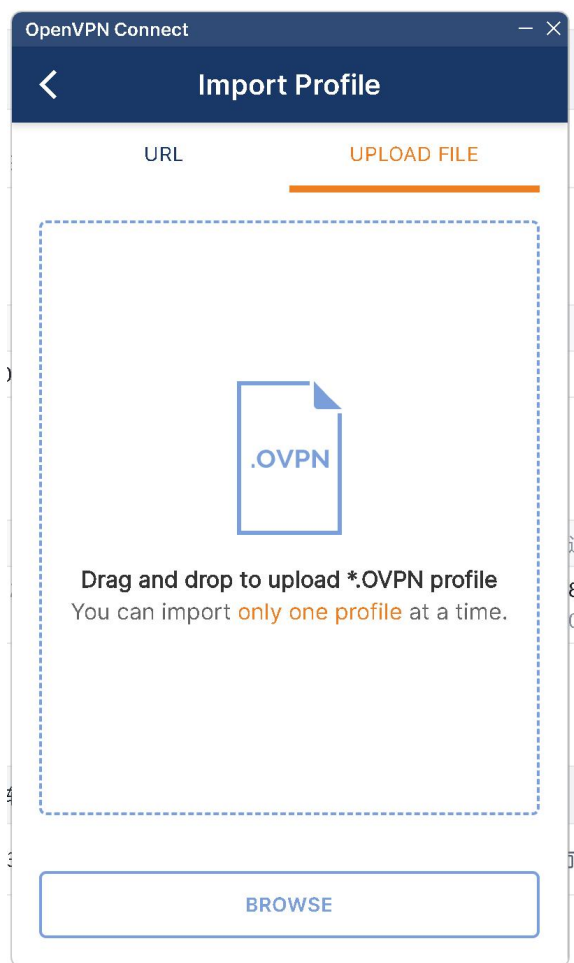


导入文件



将 zy.ovpn 文件拉入

|         |                |              |      |
|---------|----------------|--------------|------|
| zy.ovpn | 2025/5/8 19:48 | OVPN Profile | 5 KB |
|---------|----------------|--------------|------|



点击连接后，访问的内网 IP 是 <http://192.168.23.5/>

OpenVPN Connect

<

Imported Profile

Profile Name

218.90.199.133 [zy]

Server Hostname (locked)

218.90.199.133

PROFILES

CONNECT

